NoisiQ: Noise-Aware Quantum Circuit Simulation and Visualization

David Saxum & Teddy Jones
Advised by Dr. Mohamad Niknam & Dr. Richard Ross
Final Report to Satisfy Completion of QNTSCI 412 Spring 2026

Abstract

In the Noisy Intermediate-Scale Quantum (NISQ) era, circuit performance is constrained by overall error rates and the specific physical characteristics of the underlying noise. NoisiQ is an open-source Python library designed to bridge the gap between quantum circuit simulators and hardware behavior through noise-aware simulation and visualization. The package dynamically routes between scalable Clifford/Pauli propagation using the stabilizer formalism and exact density-matrix simulation to model a broad suite of errors, including Pauli noise, T1 relaxation, T2 dephasing, coherent over-rotations, and correlated multi-qubit errors. NoisiQ translates these noise parameters into visual diagnostics, such as interactive heatmaps, step-by-step error propagation animations, and multi-shot error summaries, allowing users to track how errors accumulate across gates, qubits, measurements, and circuit depth. The library also provides built-in noise suppression techniques, including dynamical decoupling, while integrating with established frameworks such as Qiskit and Stim. Designed initially as an educational and exploratory research tool, NoisiQ helps early researchers connect physical noise models, circuit-level behavior, and representative performance metrics in a single workflow to easily visualize and build intuition for quantum algorithms.

1 Introduction

The current generation of quantum computing devices, commonly referred to as Noisy Intermediate-Scale Quantum (NISQ) devices [1], represents a pivotal era in the development of practical quantum technology. Despite rapid advances in qubit counts and fabrication techniques, meaningful quantum computation remains fundamentally constrained by hardware noise. The dominant error sources are characterized by energy relaxation and dephasing, commonly, T_1 and T_2 coherence times, imperfect gate implementations, qubit-to-qubit crosstalk, and state preparation and measurement (SPAM). All collectively degrade circuit fidelity in ways that are both hardware-specific and deeply sensitive to circuit structure [2, 3].

Classical simulation serves as an essential tool for developing, validating, and benchmarking quantum algorithms prior to execution on physical hardware. However, exact classical simulation of quantum systems is inherently limited by the exponential growth of the Hilbert space: an n-qubit system requires 2^n complex amplitudes, rendering statevector simulation tractable only up to approximately ∼XXX qubits on common hardware. As counts grow into the tens and hundreds of qubits [4, 5], this exponential scaling increasingly separates the classical simulation environment from the hardware. A reality that researchers must navigate, motivating the use of approximate noise models and efficient simulation methods.

Among such is Pauli twirling, which is a technique for converting coherent gate errors into stochastic Pauli channels, enabling the use of stabilizer-based simulators. A convenient tool for circuits that would otherwise require computationally expensive density-matrix treatment [6, 7]. In addition to Pauli twirling, researchers have developed a range of methods to approximate the error processes present on real hardware to forms that can be simulated via classical methods. Nevertheless, a persistent gap in the current simulation landscape is the absence of tools that connect the physical noise models to an observable, circuit-level effect in an accessible and transparent way.

Existing production-grade simulators, such as Qiskit Aer [8] and Stim [9], are powerful and highly optimized for computational throughput; however, they are not primarily designed for educational accessibility or noise transparency. Translating a physical noise parameter—such as a T_1 relaxation time or imperfect gate implementation into an observable effect on a specific circuit requires specific customization beyond what either of these frameworks natively provide. Furthermore, a meaningful comparison of noise behavior across different quantum hardware modalities remains difficult within a single, unified workflow. For researchers entering the field and students developing physical intuition for quantum error processes, this constitutes a meaningful educational and practical gap.

To address these limitations, we introduce **NoisiQ**, an open-source Python library for noise-aware quantum circuit simulation and visualization. The library is designed to make the relationship between physical noise parameters and their circuit-level consequences transparent and accessible: it combines a Clifford-stabilizer backend, leveraging Stim's `TableauSimulator` for Pauli noise channels, with a trajectory-based density-matrix backend

for circuits involving non-Clifford operations or non-Pauli noise processes, with backend selection determined automatically from the circuit and noise model at hand. Together, these backends support a broad range of physically motivated error models, including Pauli noise, T_1 and T_2 decoherence, correlated multi-qubit errors, and mid-circuit measurements with classical feedback.

As a research instrument, it enables rapid prototyping and benchmarking of noise mitigation strategies across diverse noise regimes and hardware modalities. As an educational resource, it is designed to lower the barrier for students and early-career researchers by building physical intuition for how noise propagates through quantum circuits and interacts with algorithmic structure, a connection that existing tools do not make readily accessible.

sectionDesign Philosophy

1.1 Design Principles

Inspired by the design principles of frameworks like Qiskit, NoisiQ is built upon a foundation of modularity, extensibility, and user-centric design. The primary goal is to bridge the gap between quantum algorithms and the physical realities of hardware noise. To achieve this, NoisiQ's architecture seeks to abstract the algorithms and circuit building to focus on visualization tools that help the user build intuition, creating an environment meant to be tinkered with for research and education.

A core idea of NoisiQ's design is decoupling the Intermediate Representation (IR) from the execution backends and noise models. The built-in method for defining circuits is with the `Circuit` and `Operation` classes, which simply record the sequence of gates, measurements, and target qubits, which should feel familiar to users of Qiskit or Stim. This separation enables a dynamic routing system via the `BackendSelector`, which automatically analyzes the circuit's gate set and applied noise models to select the most efficient simulation engine.

Furthermore, this modularity ensures that NoisiQ is extensible to future quantum hardware. As quantum research evolves, users can easily introduce new, complex noise channels (such as correlated multi-qubit errors or hardware-specific cross-talk) or plug in entirely new simulation backends without needing to alter the core circuit logic or the visualization pipeline.

1.2 Teaching Tool

Quantum error correction and noise propagation are notoriously difficult concepts to grasp through mathematics alone. NoisiQ was fundamentally designed to be a visual aid in the learning process, serving as an intuitive educational tool for students and early researchers entering the field. Through interactive, step-by-step animations, users can watch in real-time as errors are injected and propagate through gates and measurements. This allows learners to go from understanding how a single Pauli error propagatesthrough a circuit to the macro-level with multi-shot heatmaps to visualize aggregate error rates. By providing this interactive visual feedback loop, NoisiQ acts as a sandbox that helps users build a physical intuition for how and why quantum algorithms fail in the NISQ era.

1.3 Straightforward Import / Export

For a new tool to be widely adopted, it must integrate seamlessly with the tools researchers already use. NoisiQ was built to complement the existing quantum software stack rather than replace it. To prevent users from having to rewrite their existing algorithms, we built NoisiQ with robust import capabilities, allowing it to parse OpenQASM 2.0 [10] and directly import circuits from established frameworks like Qiskit [8] and Stim [9].

Equally important is the ability to share the output of the library. Because research and education rely heavily on communication, NoisiQ was designed to export its visualizations into universally accessible formats. Users can easily export their circuit animations and heatmaps as MP4 videos, GIFs, and image files. This ensures that the visual intuition generated by NoisiQ can be effortlessly embedded into research papers, presentations, and educational websites, extending its utility far beyond the Python environment.

1.4 Hardware Comparison

Because different quantum architectures exhibit fundamentally different noise characteristics and connectivity constraints (i.e. superconducting qubits with fast gates but short coherence times versus trapped ions with long coherence but slower gates), an algorithm optimized for one platform may perform poorly on another. NoisiQ was designed to make cross-platform hardware comparison as effortless as possible. By decoupling the circuit definition from the physical noise models, NoisiQ allows users to write an algorithm once and simulate it across a variety of pre-loaded hardware profiles, including IBM's Heron processors [11, 12], Quantinuum's H2 systems [13, 14], and IonQ's Forte [15, 16].

Rather than manually configuring complex noise channels, users can simply select a target device, and NoisiQ automatically applies the appropriate T_1 and T_2 times, gate infidelities, SPAM errors, and architecture-specific phenomena like ZZ crosstalk or spectator errors. When combined with the library's visual diagnostics, this allows users to see exactly how and why an algorithm fails on different devices. For example, users can make an interactive heatmap to compare the impact of coherent overrotations on a trapped-ion system versus the rapid decoherence of a superconducting processor. This capability empowers students and researchers to perform direct comparative benchmarking, build intuition for hardware-specific failure modes, and identify the most suitable

hardware modality for their quantum applications.

2 Simulation Architecture

The core simulation architecture of NoisiQ is built around a dynamic routing system that selects the most efficient backend for a given circuit and noise model. Because quantum simulation is computationally expensive, simulating every circuit with full density matrices is unfeasible for deep algorithms or large qubit counts. To solve this, NoisiQ implements a `BackendSelector` that inspects the Intermediate Representation (IR) before execution. It analyzes the gate set (e.g., Clifford vs. non-Clifford) and the noise channels (e.g., Pauli vs. non-Pauli) to hand off the simulation to the optimal engine.

2.1 Simulator Handoffs

For circuits composed entirely of Clifford gates (e.g., H, S, CNOT) and Pauli noise, NoisiQ routes execution to the `StimTableauBackend`. Stim is a stabilizer code simulator that tracks the evolution of Pauli frames rather than full state vectors. By leveraging the Gottesman-Knill theorem [17], Stim scales polynomially with number of qubits rather than exponetially, giving it the ability to simulate thousands of qubits and millions of gates in milliseconds. This backend is heavily utilized in NoisiQ for large-scale Quantum Error Correction (QEC) demonstrations and multi-shot error tracking where non-Clifford gates are not required.

When a circuit contains non-Clifford operations (like T gates or arbitrary rotations) or complex non-Pauli noise models (like coherent over-rotations or amplitude damping), the stabilizer formalism breaks down. In these cases, NoisiQ automatically hands off execution to the `TrajectoryBackend` or the `QiskitAerBackend`.

The `TrajectoryBackend` handles non-Pauli noise by simulating the noise channels as discrete Kraus operators or unitaries, applying them shot-by-shot to a statevector. For exact density matrix simulation of universal circuits, NoisiQ integrates directly with Qiskit Aer. By compiling the NoisiQ IR down to a `QuantumCircuit` and converting our custom Kraus channels into Qiskit `quantum_error` objects, we leverage Qiskit Aer's highly optimized density matrix simulator. This handoff allows NoisiQ to maintain exact physical accuracy for complex noise models on smaller qubit counts (typically ≤ 13 qubits) while seamlessly scaling up via Stim when the gate set and error model allow it.

2.2 Noise Types

A major goal of NoisiQ is to accurately reflect the physical noise profiles of quantum hardware. The library includes pre-loaded hardware profiles (e.g., IBM Heron, Quantinuum H2) that automatically map device specifications to the following noise channels:

2.2.1 Pauli Errors and Pauli-Twirling

At the core of scalable quantum error simulation (with Stim) is the Pauli error channel, which models stochastic bit-flip (X), phase-flip (Z), and combined (Y) errors. In NoisiQ, users can inject custom noise into their circuits using the `PauliError` class, or use convenience functions like `depolarizing_error` and `dephasing_error` to apply standard noise models to specific gates.

Because Pauli errors map perfectly to the stabilizer formalism, they are fully compatible with NoisiQ's `StimTableauBackend`, allowing for the simulation of massive circuits in polynomial time. To take advantage of this scalability when modeling non-Pauli hardware noise (like T_1, T_2, or coherent over-rotations), NoisiQ includes a `representation="pauli_twirl"` mode in its hardware profiles. This feature automatically approximates continuous, non-Pauli physical errors into their first-order stochastic Pauli equivalents. For example, a coherent rotation of angle ϵ around the Z-axis is "twirled" into a discrete Z error occurring with probability $\sin^2(\epsilon)$. This bridging mechanism gives users the flexibility to choose between exact physical simulation on small circuits or fast, large-scale approximations on deep circuits.

2.2.2 Amplitude Damping (T_1)

Amplitude damping is the energy relaxation where a qubit in the $|1\rangle$ state spontaneously decays to $|0\rangle$. This is implemented via a `KrausChannel` parameterized by the hardware's T_1 time and the duration of the gate or idle period.

2.2.3 Phase Damping (T_2)

Phase damping models the loss of transverse coherence (dephasing) without energy loss. It is parameterized by T_2 and the gate time, gradually shrinking the off-diagonal elements of the density matrix.

2.2.4 Coherent Over-rotation

Unlike stochastic errors that destroy purity, coherent errors are small, systematic unitary rotations (e.g., $U_{\text{err}} = \exp(-i\epsilon Z)$) caused by miscalibrated control pulses. Because these errors accumulate coherently (amplitudes add linearly, probabilities scale quadratically), they are highly detrimental in deep circuits. NoisiQ models this exactly via the `CoherentRotation` channel, forcing a handoff to the `TrajectoryBackend` to capture the quadratic accumulation that standard Pauli-twirling [18, 19] misses.

2.2.5 Crosstalk and Correlated Errors

On real hardware, errors are rarely independent. NoisiQ implements `CorrelatedPauliError` channels to model joint failure modes. For example, always-on residual coupling in superconducting qubits causes joint ZZ phase errors during idle times, and addressing beam

spillover in trapped ions causes spectator errors on neighboring qubits during two-qubit gates.

2.2.6 SPAM Errors

State Preparation and Measurement (SPAM) errors are modeled by injecting depolarizing noise at the boundaries of the circuit, with state preparation errors applied before the first gate and measurement errors applied after the final gate, ensuring that the initialization and readout fidelities match published hardware benchmarks.

3 Visualization and Diagnostics

NoisiQ contains three visualization outputs that are built directly on top of the simulation result objects returned by the backends described in Section 2: a step-by-step error trajectory animation, a static aggregate error heatmap, and an interactive diagnostic interface. All three share a common data path, there is need for additional post-processing or format conversion between simulation and visualization and each can be exported to a static format or used inside of a Jupyter environment.

3.1 Error Trajectory Visualization

The error trajectory visualization provides a gate-by-gate, step-through view of how Pauli errors accumulate and propagate through a circuit during simulation. It is implemented in the `CircuitAnimator` class, which accepts either a single-shot `StimTableauResult` or a many-shot `AggregateResult` and renders an interactive widget in Jupyter via `ipywidgets`, with playback controls including Play/Pause, step-forward, step-back, and a frames-per-second slider. The animator can also be exported as a `matplotlib` `FuncAnimation` object for GIF or PNG filmstrip output via the library's `export` module.

Single-shot mode. In single-shot mode, the animator steps through the exact Pauli frame computed by the `StimTableauBackend` at each circuit layer. At each timestep t, the cumulative Pauli operator on each qubit wire is displayed as a floating label, and the right-margin annotation reports the final Pauli string across all qubits together with the total number of error events injected during the shot.

Figure 1 shows layer-by-layer Pauli-frame snapshots from a single-shot run on a 3-qubit circuit, tracing an injected X error from its point of origin through forward propagation, syndrome detection, and conditional correction. A minimal runnable example is provided in Appendix .5.

Many-shot aggregate mode. In many-shot mode, the modal error trajectory is reconstructed from the `counts_by_pauli` distribution in the `AggregateResult`. The right-margin annotation reports the empirical per-qubit error rate (the fraction of shots in which any error occurred on that wire), the zero-error fraction, and the total shot count. This mode is the primary diagnostic for understanding how a circuit behaves at scale, where single-trajectory inspection is uninformative.

3.2 Aggregate Error Heatmap

The aggregate error heatmap renders the full circuit diagram with the possibility of two superimposed visual error indicators: Gaussian halo overlays on gate boxes encoding both the gate's own per-shot error contribution and its expected downstream Pauli propagation burden, and soft halo gradients on idle wire segments scaled by T_1/T_2 decoherence accumulated during qubit idle time. Both indicators are derived directly from the noise model and simulation result. The magnitude of gate halos depends on the noise model supplied: a depolarizing Pauli model and a full hardware profile with calibrated gate error rates and coherence times will generally produce different absolute burden values for the same circuit. However, because halo intensities are normalized relative to the highest-burden gate in each run, two circuits with very different underlying error rates can produce visually similar heatmaps. The colorbar tick labels and the per-gate burden values surfaced by the interactive diagnostic interface (Section 3.3) expose the true absolute magnitudes, and should be consulted alongside the visual when comparing circuits or noise models.

Gate halos: per-gate error contribution and downstream propagation burden. For each gate k with attached noise channel $\mathcal{N}_k$, the expected downstream error burden is defined as:

$$b_k = \sum_{E \neq \mathbf{I}} p(E \mid k) \cdot \big(1 + \sigma(E, k)\big) \tag{1}$$

where $p(E \mid k)$ is the probability of Pauli outcome E from channel $\mathcal{N}_k$, and $\sigma(E, k)$ is the number of subsequent gates that E intersects after propagating forward through the remaining circuit under Clifford conjugation rules. The factor $(1+\sigma)$ accounts for both the originating gate itself and every downstream gate the error contaminates. Two-qubit gates generally produce higher burden scores than single-qubit gates: the sum in Eq. 1 runs over all non-identity joint Pauli outcomes on both qubits, and the larger gate error rates typical of two-qubit operations in hardware noise models further amplify b_k relative to single-qubit counterparts. For mid-circuit measurements, $\sigma(E, k)$ is evaluated under a QEC-corrected mode, a worst-case mode, or a weighted average, as described in Section 2.

The visual intensity of each gate halo is normalized to the range $[\alpha_0, 1]$:

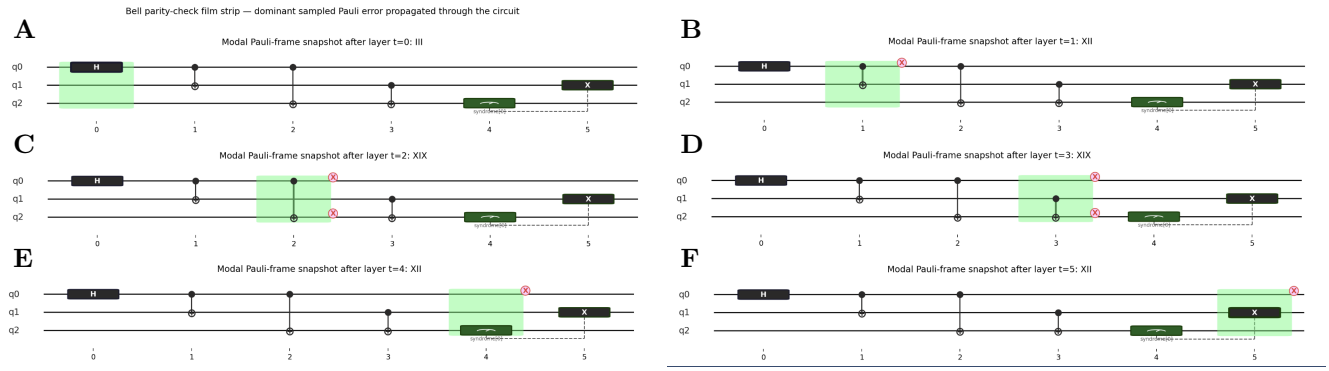

Figure 1: Layer-by-layer Pauli-frame snapshots for a single-shot run on a 3-qubit circuit, exported as a static filmstrip from the `CircuitAnimator`. At each timestep, the operation time is highlighted with a green box that advances through the circuit left to right, and the current Pauli error operator is carried as a floating label just above the affected qubit wire. **(A)** Initial circuit layer: no active errors; all qubit wires are in the identity frame. **(B)** X error injected on the data qubit; the Pauli label appears on the affected wire. **(C–D)** The X error propagates forward through subsequent entangling gates, spreading to ancilla qubits as the Clifford conjugation rules transform the Pauli frame. **(E)** Mid-circuit syndrome measurement fires; the classical outcome signals the presence of the error, and the conditional correction is queued. **(F)** Conditional correction applied; the Pauli frame returns to the identity, confirming successful error detection and correction.

$$\tilde{b}_k = \alpha_0 + (1 - \alpha_0) \cdot \frac{b_k}{\max_j b_j} \tag{2}$$

where $\alpha_0 = 0.35$ is a minimum intensity floor applied to every gate carrying a non-zero error channel, regardless of its downstream propagation score. This floor is intentional: a gate at the end of a circuit has no downstream gates to contaminate ($\sigma = 0$), so its raw burden b_k reduces to the channel event probability alone. Without a floor such a gate would appear visually clean despite being genuinely noisy, misrepresenting the true error budget to the user. The floor ensures that any gate with an attached noise channel receives a visible halo, while the relative scaling above α_0 preserves the propagation burden ordering across the circuit. Absolute and logarithmic normalization modes are also available for cross-circuit comparison on a fixed scale.

Wire halos: idle-time decoherence via Pauli-twirl proxy. Idle qubit segments between active gates accumulate T_1 relaxation and T_2 dephasing. To represent these physically-motivated Kraus channels within the same Pauli-frame visualization, NoisiQ converts them to stochastic Pauli channel proxies.

For energy relaxation (amplitude damping) over idle duration Δt with relaxation time T_1, the decay probability is:

$$\gamma = 1 - e^{-\Delta t / T_1} \tag{3}$$

which is mapped to a symmetric depolarizing Pauli proxy $p_X = p_Y = p_Z = \gamma/4$. For dephasing over idle duration Δt with dephasing time T_2:

$$\lambda = 1 - e^{-2\Delta t / T_2} \tag{4}$$

which maps to a pure phase-flip proxy $p_Z = \lambda/2$, $p_X = p_Y = 0$. For coherent over-rotation errors of the form $U_{\text{err}} = \exp(-i\,\varepsilon\,P)$ about a Pauli axis P, the twirled stochastic approximation assigns:

$$p_P = \sin^2(\varepsilon) \tag{5}$$

The combined idle-segment halo intensity is then:

$$I_{\text{wire}} = 1 - (1 - \gamma)(1 - \lambda) \tag{6}$$

and is rendered as a soft vertical Gaussian gradient along the wire between the bounding active gates. The wire halos use a separate blue colormap (dark navy at negligible decoherence to match the wire, bright cyan at maximum) and are accompanied by a dedicated idle-decoherence colorbar at the bottom of the figure, annotated with the observed $T_1\,\gamma$ and $T_2\,\lambda$ component ranges across the circuit.

Figure 2 presents the aggregate error heatmap for a Bell parity-check circuit rendered by `plot_error_heatmap()`. The `[qec]` rendering mode activates correction-aware burden propagation: when the syndrome ancilla (q2) measures `1`, the `c_if` X(`q1`) correction is assumed to fire and *cancel* the propagating error, so the heatmap reports the *residual* downstream burden after the parity check does its job. As a result, gates occurring before the ancilla can act—the Bell preparation H and CNOT at t=0–1—carry the highest gate burden, while the ancilla CNOT pair at t=2–3 has its downstream spread partially neutralized by the correction.

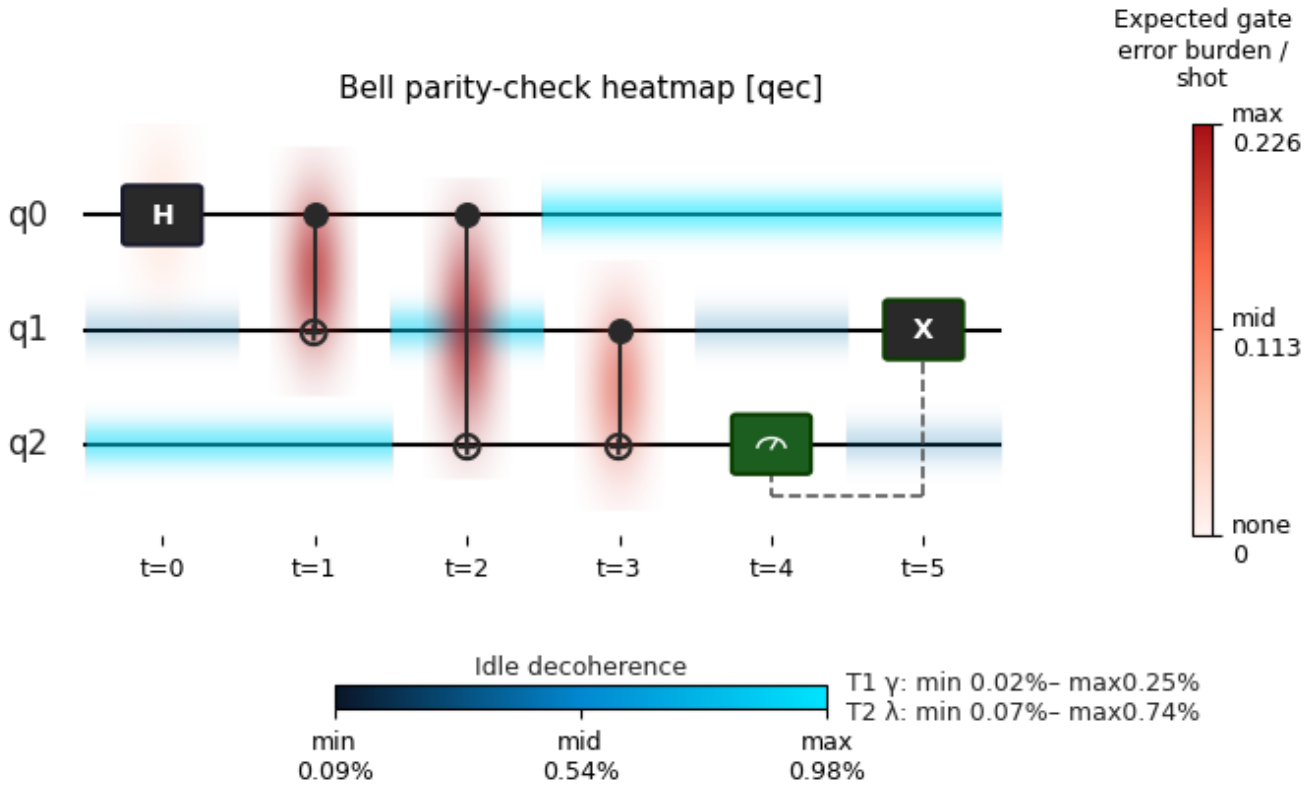

Figure 2: Aggregate error heatmap for a Bell parity-check circuit rendered in `[qec]` correction-aware mode under an `ibm_eagle_r3` hardware noise profile aggregated over 4 096 shots with a Pauli-twirled noise model. **Gate halos** (red scale) show the expected downstream Pauli propagation burden and attached gate noise channel via (Eq. 1). In `[qec]` mode the burden walk assumes the mid-circuit correction fires correctly, so only errors that precede or evade the ancilla contribute to the displayed burden. **Wire halos** (blue scale) show the idle-time decoherence intensity I_{wire} (Eq. 6)
This circuit example represents a bit-flip parity repair only; not full Bell-state QEC.

3.3 Interactive Diagnostic Interface

The aggregate heatmap provides a circuit-level summary of noise burden and decoherence. However, it's also important to isolate the specific time operation or circuit element that drives a large error event, visually displayed by a high-intensity halo, requires access to the underlying values at that circuit element. The `interactive_heatmap()` function extends the static heatmap with spatially resolved readout system. Where every element of the circuit diagram: gate boxes, idle wire segments, qubit end-caps, and measurements, is turned into to a clickable or hoverable region, and selecting any element displays a formatted readout of the element, its noise parameters and simulation statistics. The function exposes two rendering modes through the `display_mode` parameter, 3 and 4 illustrate these two modes applied to a 3-qubit Bell parity check circuit.

- **Panel mode** (`"panel"`): the figure canvas and an `ipywidgets.Output` widget are composed side-by-side inside an `HBox` container, and a `button_press_event` callback repopulates the panel on each click with a full-width aligned readout. This mode requires the `ipympl` backend; if a non-interactive backend is active, the function raises a descriptive error.

- **Annotate mode** (`"annotate"`): the readout is rendered as a floating tooltip constructed via `ax.annotate` and refreshed on each `motion_notify_event`. This mode is compatible with any Matplotlib backend and is the appropriate choice when `ipympl` is unavailable or when a compact, cursor-following summary is preferred over a persistent side panel.

To resolve which circuit element the cursor is over, a spatial dispatch layer maps each element to a bounding box defined by the axis coordinates of the circuit layout. In `"panel"` mode, the cursor coordinates are tested against these boxes in priority order to select the readout type:

- **Gate**: reports the gate name, qubit targets, and timestep, followed by the error channel assigned to that operation. For Pauli channels this includes the individual probabilities p_X, p_Y, p_Z, and their sum p_Σ; for amplitude-damping and dephasing channels, the damping parameter γ and dephasing parameter λ are shown instead. The readout then lists per-qubit cumulative decoherence accumulated before and through the gate, followed by the many-shot statistics, such as total error events per shot, drawn from the `AggregateResult`.

- **Measurement**: measurement operations occupy the same layout region as gate boxes but trigger a dedicated readout branch. Rather than Pauli channel parameters, the panel reports the cumulative bit-flip probability p_{wrong} on the measured qubit, which aggregates all X- and Y-type error channels applied to that qubit up to the measurement time and represents the probability of a corrupted classical readout.

- **Wire segment**: reports the segment's T1 damping γ, T2 dephasing λ, and combined decoherence intensity $I_{\text{wire}} = 1 - (1-\gamma)(1-\lambda)$, together with the cumulative T1 and T2 decoherence on that qubit integrated from the circuit start to the cursor's position within the segment, providing a time-resolved view of decoherence buildup between gates.

- **Qubit end-cap**: summarizes the full-circuit cumulative T1 and T2 decoherence on that qubit together with the per-qubit error event count and error rate from the many-shot run.

The downstream burden b_k and visual halo intensity $\tilde{b}_k$ reported in the gate readout are produced by the same computation used to generate the heatmap coloring, so the colorbar scale and the panel's numerical values are consistent by construction regardless of the chosen `impact_metric`. An optional argument accepts a metadata dictionary containing information such as the hardware profile source, noise representation, and run-level summary statistics; this metadata is forwarded into all four circuit element types, allowing the panel to serve as a record for the full simulation context.

4 Metrics

To provide a comprehensive understanding of circuit performance under various noise models, NoisiQ implements a suite of mathematical diagnostics along with the visual tools. These metrics allow users to quantify the degradation of quantum information, from state comparisons to hardware benchmarks.

4.1 Fidelity Calculations

The primary measure of success for a quantum circuit is fidelity, which quantifies how close the noisy output is to the ideal target state [20]. Because NoisiQ employs a dynamic routing architecture, fidelity is calculated differently depending on the simulation backend. For exact density matrix simulations (Qiskit Aer), the library computes the standard state fidelity by comparing the ideal output state with the noisy density matrix ρ. Assuming the ideal output is a pure state $|\psi\rangle$, this simplifies to:

$$F = \langle\psi|\,\rho\,|\psi\rangle \tag{7}$$

In cases where the target is a mixed state σ, NoisiQ evaluates the generalized mixed-state fidelity:

$$F(\rho,\sigma) = \left(\operatorname{Tr}\sqrt{\sqrt{\rho}\sigma\sqrt{\rho}}\right)^2 \tag{8}$$

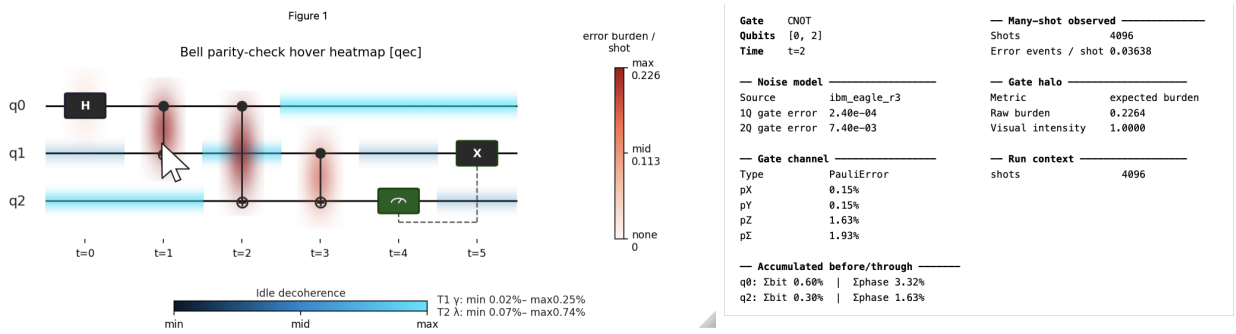

Figure 3: Panel mode of the interactive diagnostic interface. **Left**: the aggregate error heatmap with gate and wire halos. **Right**: the `ipywidgets.Output` panel populated by clicking the cursor over the CNOT gate on qubits [0, 2] at $t = 2$ (the cursor sits atop the represented gate location), showing the assigned Pauli channel probabilities p_X, p_Y, p_Z, per-qubit cumulative decoherence, many-shot zero-error survival, and the downstream burden b_k and halo intensity $\tilde{b}_k$ (Eq. 1, Eq. 2) that determine the gate's color in the heatmap.

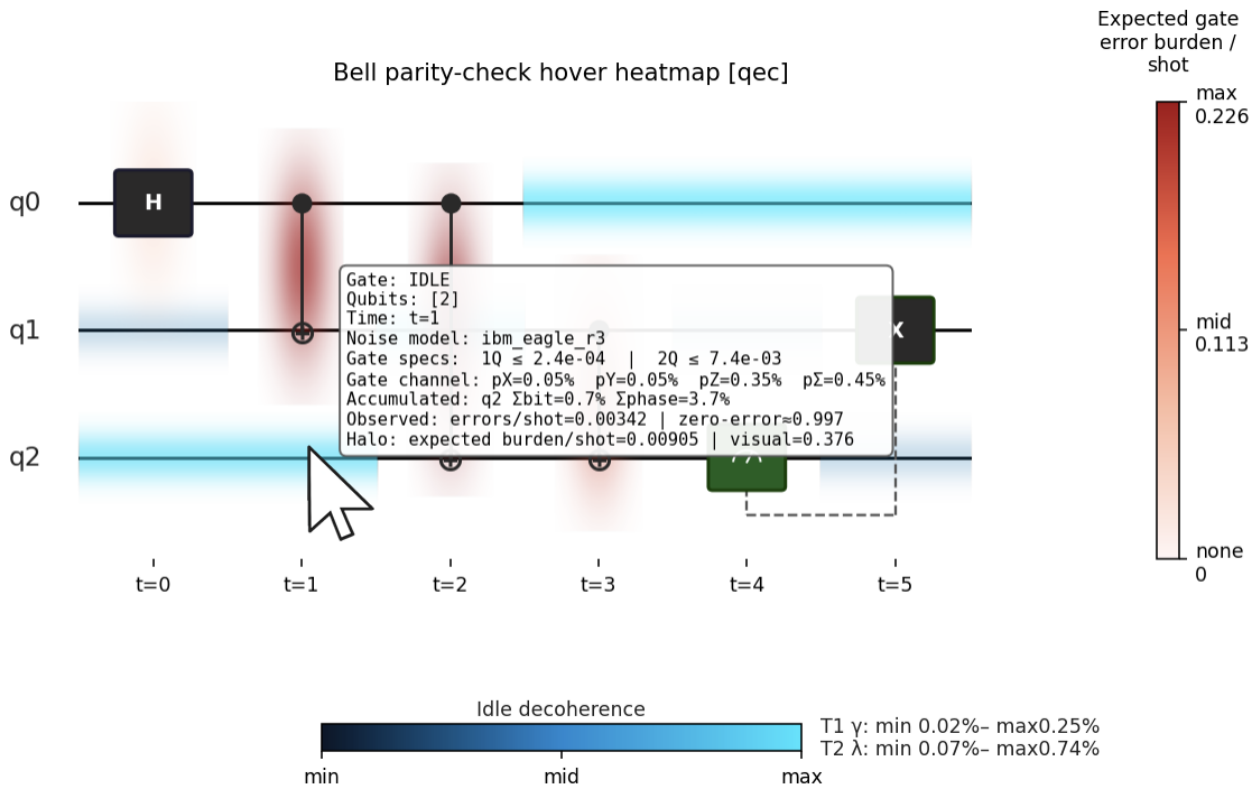

Figure 4: Hover (tooltip) mode of the interactive diagnostic interface. Hovering the cursor over an element (here the idle interval on qubit [2] at $t = 1$, with the cursor sitting atop the represented location) raises an inline tooltip reporting the gate type and timestep, the noise-model gate-error bounds, the assigned Pauli channel probabilities, per-qubit accumulated bit/phase decoherence, the many-shot observed errors-per-shot and zero-error survival, and the downstream burden b_k and halo intensity $\tilde{b}_k$ (Eq. 1, Eq. 2) that determine the element's color in the heatmap.

When working with large Clifford circuits via the `StimTableauBackend`, constructing the full state vector isn't computationally feasible. While the stabilizer formalism (and Stim) allow for exact wave function overlap calculations between pure tableaus, NoisiQ instead optimizes multi-shot noisy simulations by tracking Pauli error frames via a Monte Carlo approach. The library determines the fraction of simulation shots where the net propagated Pauli error belongs to the target state's stabilizer group. Because applying a stabilizer group element leaves the target state unchanged, this serves as a mathematically rigorous and highly scalable proxy for state fidelity.

4.2 State Purity

Purity quantifies how much quantum information has been lost to the environment through decoherence. NoisiQ calculates global purity by taking the trace of the density matrix squared:

$$\gamma = \mathrm{Tr}\big(\rho^2\big) \tag{9}$$

This value ranges from 1 for a perfectly pure state down to $1/2^n$ for a completely mixed n-qubit state.

The library also tracks the purity of individual qubits, where a value of 0.5 represents a maximally mixed state. However, this metric requires context depending on the circuit's structure. In highly entangled systems, such as a Greenberger-Horne-Zeilinger (GHZ) state, tracing out each qubit of the system will intentionally result in purities of around 0.5 for each qubit. Therefore, individual qubit purity is most informative in circuits that uncompute back to a separable initial state, allowing users to isolate which physical qubits suffered the most decoherence during execution.

4.3 Zero-Error Survival

NoisiQ also provides operational benchmarks that reflect the practical realities of cloud-based quantum computing. One such metric is the zero-error survival rate, defined as the fraction of total shots that execute without a single error occurring.

$$P_0 = \frac{N_{\text{no errors}}}{N_{\text{total}}} \tag{10}$$

This metric is relevant because most hardware platforms rely on post-selection and error mitigation techniques that discard runs where errors are detected. If an algorithm exhibits a low zero-error survival rate, the overhead required to successfully execute and post-select the results will scale dramatically. By tracking this metric, NoisiQ allows users to estimate the resource overhead and cloud budget efficiency of their algorithms before deploying them to the hardware platforms.

4.4 Process and Average Circuit Fidelity

For scalable simulations utilizing the `StimTableauBackend`, we use a different fidelity metric for circuits meant to take arbitrary states as inputs. Unlike state fidelity, which forgives errors that act trivially on a specific target state (such as a Z error on a $|0\rangle$ state), NoisiQ tracks the process fidelity, which strictly requires the net error channel to be the Identity operator.

Using this process fidelity (F_{pro}), we derive the average circuit fidelity over all possible input states. For an n-qubit system with dimension $d = 2^n$, this is calculated using the standard dimension formula [21]:

$$F_{\text{avg}} = \frac{d \cdot F_{\text{pro}} + 1}{d + 1} \tag{11}$$

This metric is useful for characterizing the overall quality of a noisy Clifford circuit independent of the data qubits being processed.

5 NoisiQ Examples

Including the Bell parity check example with the three following examples each exercise a distinct capability of the NoisiQ library: mid-circuit measurement and QEC-style post-selection (Section 3.2), hardware noise profile integration (Section 5.1), built-in noise suppression techniques (Section 5.2), and automatic backend routing for non-Clifford circuits (Section 5.3).

5.1 Greenberger–Horne–Zeilinger (GHZ) State

The Greenberger–Horne–Zeilinger state $|\mathrm{GHZ}_N\rangle = (|0\rangle^{\otimes N} + |1\rangle^{\otimes N})/\sqrt{2}$ is prepared by a single Hadamard gate on qubit q_0 followed by a cascade of $N-1$ CNOT gates that spread entanglement to q_1 through q_{N-1}. The circuit is entirely Clifford, so `ManyShotRunner` routes directly to the Stim backend for the aggregate pass; a separate density-matrix pass via `TrajectoryBackend` is used for fidelity and purity metrics at system sizes up to 13 qubits. GHZ state fidelity is a standard cross-platform benchmark that has been reported across multiple hardware systems at varying qubit counts, making it a convenient reference for evaluating the accuracy of NoisiQ's hardware noise profiles.

Error propagation through a GHZ circuit is particularly sensitive to two-qubit gate errors: a single CNOT fault anywhere in the cascade flips the parity of every subsequent qubit, so the zero-error fraction drops rapidly with circuit depth and qubit count. This makes the GHZ example a useful test of whether the noise model parameters produce a degradation trajectory that is consistent with published hardware results.

Simulations are run under an `ibm_eagle_r3` hardware noise profile with a Pauli-twirled noise model aggregated

over 4 096 shots. The metrics reported include GHZ state fidelity $F = \langle \mathrm{GHZ}_N | \rho | \mathrm{GHZ}_N \rangle$, global purity $\mathrm{Tr}(\rho^2)$, GHZ subspace population, branch coherence, and per-qubit marginal purity. A stabilizer fidelity metric derived from the Pauli-frame aggregate pass provides a scalable estimate at large qubit counts where the exact density matrix cannot be computed.

The fidelity comparison serves only as a scale reference: NoisiQ simulates a hardware profile reconstructed from published noise parameters and cannot re-execute the original experimental circuits. The largest source of discrepancy is circuit size—NoisiQ exactly simulates 13 qubits, whereas the IBM device operates on 18, and those additional qubits introduce more CNOT gates and correspondingly more error channels. Differences in qubit connectivity, gate decomposition, and calibration drift between the published experiments and the simulation inputs are further expected contributors.

Figure 5 shows the family heatmap for the 13-qubit GHZ circuit.

5.2 Dynamical Decoupling

Dynamical decoupling (DD) suppresses idle-time decoherence by applying a sequence of refocusing pulses during periods when a qubit is not participating in a gate operation [22]. When a qubit is subject to a quasi-static or slowly varying Z drift, a suitably chosen pulse sequence causes the accumulated phase to reverse sign across the idle window so that it cancels, recovering coherence that would otherwise be lost. Crucially, this refocusing is a *coherent* effect: it acts on a definite accumulated phase, not on incoherent stochastic flips. NoisiQ provides a built-in `apply_dd()` function that inserts refocusing sequences into the idle gaps of a sparse circuit, and the resulting circuits are compared under a common noise model using the same `ManyShotRunner` and `TrajectoryBackend` pipeline.

Setup. An 8-qubit GHZ state is prepared, stored across 32 idle layers, and then un-prepared by the inverse circuit. The storage window is the region where decoherence accumulates and where DD pulses are inserted. The noise model applies a quasi-static Z drift of $\varepsilon = 0.07\,\mathrm{rad}$ per idle slot as a per-shot coherent over-rotation, representative of low-frequency idle dephasing on IBM Eagle r3; this regime is chosen because it is precisely where DD refocusing is physically effective.

Producing a simulation-ready circuit requires three ordered builds. The first constructs a *sparse* circuit with explicit `t=` gate-time indices and no idle operations, fixing the gate timing. The second calls `apply_dd(sparse, sequence=...)` to insert the chosen refocusing pulses into the idle gaps at their correct time positions; the pulse pattern is tiled to fill the window, so the number of pulses scales with the window length and the refocusing density stays roughly constant. The third calls `fill_idle_with_identities()` to populate the remaining idle slots with explicit `IDLE` operations, ensuring the noise model advances on every qubit between gates. Only the output of the third build is simulated; the first two are necessary intermediate representations that keep the pulse positions and idle-noise durations physically self-consistent.

Measurement metric. Because the demonstration circuit un-prepares the GHZ state, a naive fidelity to $|0\ldots0\rangle$ is measured *through* the closing Hadamard, which maps residual Z phase error to X bit-flip error ($HZH = X$) and amplifies it, ranking DD sequences backwards. We therefore report the headline fidelity as the *GHZ-state fidelity measured at the storage-window exit, before the uncompute*, $F_{\mathrm{GHZ}} = \frac{1}{2}(\rho_{0,0} + \rho_{N,N}) + \mathrm{Re}\,\rho_{0,N}$, which is directly sensitive to dephasing without the Hadamard amplification. Global purity $\mathrm{Tr}(\rho^2)$ is reported alongside it; an effective sequence recovers both F_{GHZ} and $\mathrm{Tr}(\rho^2)$ toward unity. The fidelity and purity passes use a coherent noise representation so the idle-Z drift is a genuine rotation that the pulses can refocus, while the error-burden heatmap is generated from a separate Pauli-frame many-shot pass on the full circuit.

Sequences. The universal sequences compared are summarized in Table 1. The Hahn echo (a single X per period) is *omitted*. Under the GHZ storage circuit each qubit's idle window has a different length, so a density-preserving tiling places a different, mixed-parity number of pulses on each qubit. For the universal sequences this is harmless because each period nets to the identity regardless of how many whole periods are tiled; a single-X Hahn period, however, is not self-inverse, so a non-uniform net X maps the GHZ poles $|0\ldots0\rangle$ and $|1\ldots1\rangle$ to non-complementary bitstrings and removes population from the $\{|0\ldots0\rangle, |1\ldots1\rangle\}$ subspace entirely (both branch populations fall to zero while the state stays nearly pure). Single-X Hahn is therefore ill-defined as a DD sequence for multi-qubit GHZ storage when the idle windows are not uniform, and we restrict the comparison to the universal sequences.

Results. Figure 6 shows the four circuits with their error-burden heatmaps and metric boxes. All three universal sequences substantially recover coherence relative to the undecoupled baseline: F_{GHZ} rises from 0.470 (No DD) to 0.749 (XY-4), 0.734 (XY-8), and 0.769 (CPMG), with global purity rising in step from 0.450 to 0.59–0.61. The two metrics agree in direction, confirming that the recovery reflects genuine coherent refocusing of the stored state rather than an artifact of the readout. The three universal sequences perform comparably here; their differences are within the per-shot sampling scatter of the coherent simulation at this shot count, so we do not read a strict ordering among them. The error-burden halos (red) remain concentrated on the entangling CNOT layers at

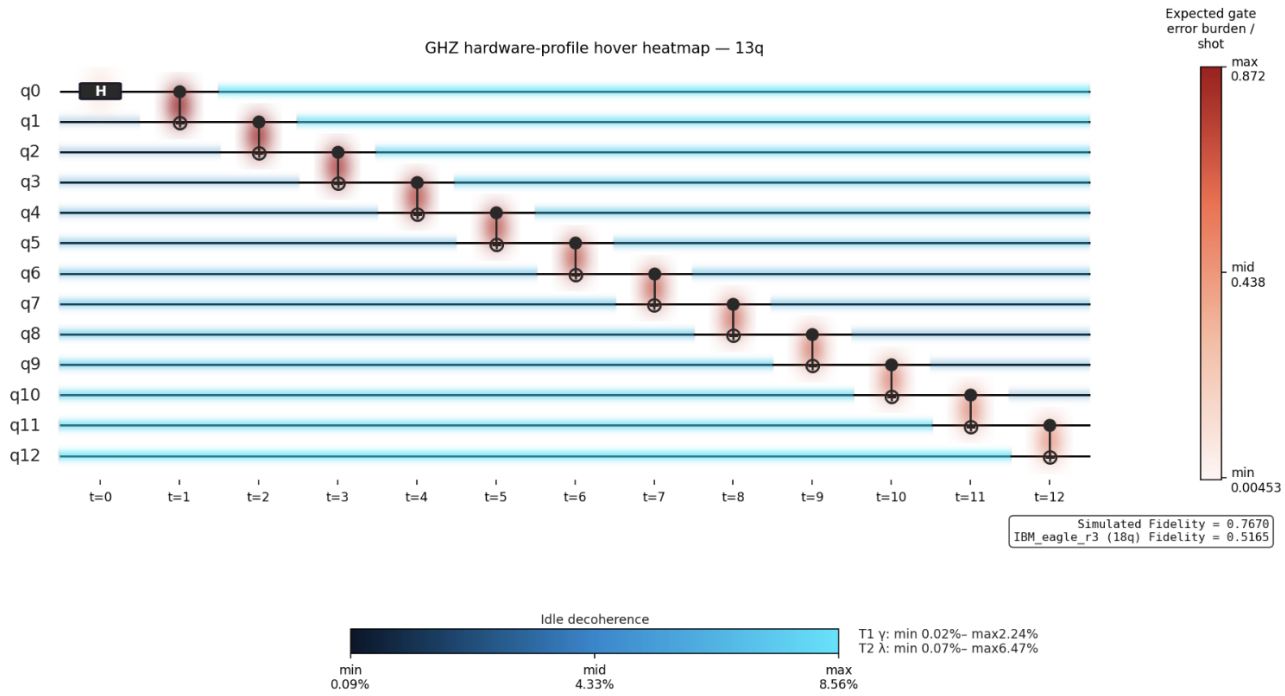

Figure 5: Aggregate error heatmap for a 13-qubit GHZ state under an `ibm_eagle_r3` hardware noise profile, aggregated over 4 096 shots with a Pauli-twirled noise model. The simulated fidelity is 0.7670, compared with a reported `ibm_eagle_r3` (18q) device fidelity of 0.5165, a difference of 0.2505.

Table 1: Dynamical decoupling sequences compared in the NoisiQ 8-qubit GHZ storage example. All sequences are applied over the same 32-layer idle window under an identical quasi-static Z-drift noise model.

Sequence	**Pulse order**	N	**Primary effect**
Baseline	—	0	No refocusing
CPMG	$X \cdot X$	2	Z dephasing; improved low-frequency rejection
XY-4	$X \cdot Y \cdot X \cdot Y$	4	Universal first-order refocusing
XY-8	$X \cdot Y \cdot X \cdot Y$ $Y \cdot X \cdot Y \cdot X$	8	Universal higher-order refocusing

the circuit ends, which lie outside the storage window and are not addressable by storage-window DD.

5.3 Quantum Fourier Transform

The quantum Fourier transform (QFT) is the quantum analog of the discrete Fourier transform and underlies a broad class of quantum algorithms including Shor's factoring algorithm and quantum phase estimation [20]. The 3-qubit QFT consists of Hadamard gates, controlled phase rotations at angles $\pi/2$ and $\pi/4$, and a final bit-reversal SWAP. NoisiQ realizes the $\pi/2$ rotation with the native controlled-S (CS) gate and decomposes the $\pi/4$ rotation into single-qubit phase gates $P(\pm\pi/8)$ and two CNOTs. Both CS (a third-level gate in the Clifford hierarchy) and $P(\pm\pi/8)$ lie outside the Clifford group, so the circuit cannot be simulated by Stim. NoisiQ's `BackendSelector` detects the non-Clifford gates and selects the trajectory-based density-matrix backend (`TrajectoryBackend`) automatically. This example is included specifically to demonstrate that routing: the user supplies a single circuit and a single noise model, and NoisiQ chooses the correct simulator with no additional configuration.

A single `TrajectoryBackend` pass over the faithful QFT yields both the aggregate per-gate error statistics that drive the heatmap (Fig. 7) and the averaged density matrix ρ, from which we report the state fidelity $F = \langle \mathrm{QFT} \,|\, \rho \,|\, \mathrm{QFT} \rangle$ and the global purity $\mathrm{Tr}(\rho^2)$. Because the trajectory backend imposes no Clifford restriction on the gate set, no Clifford stand-in circuit is needed: the heatmap is rendered directly from the true QFT.

6 Conclusion

In this report, we introduced **NoisiQ**, an open-source Python library designed to bridge the gap between abstract quantum algorithms and the physical realities of hardware noise in the NISQ era. While existing

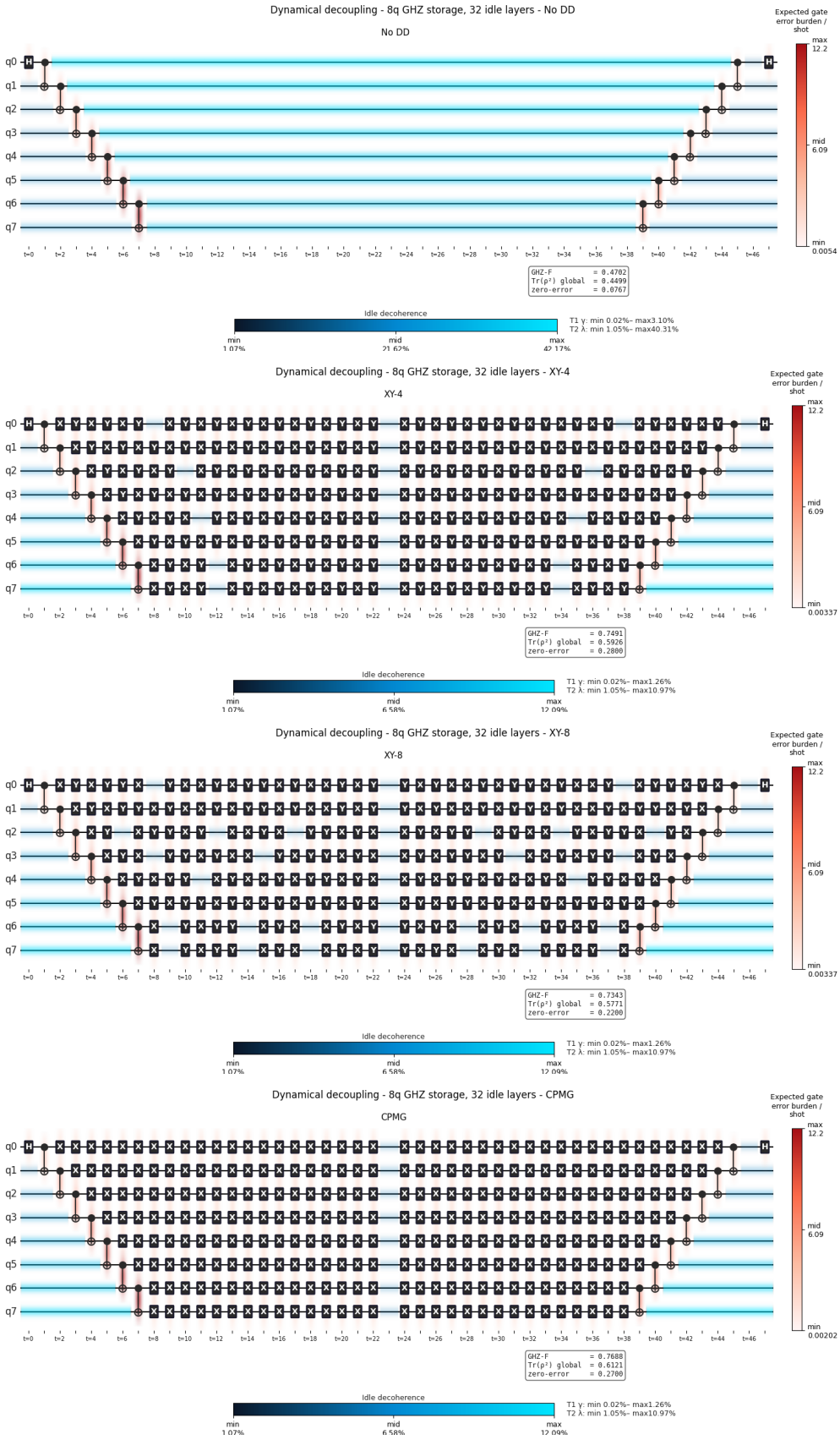

Figure 6: Dynamical decoupling on an 8-qubit GHZ storage circuit (32 idle layers) under an `ibm_eagle_r3` noise profile with quasi-static idle-Z drift. The panels show the No DD, XY-4, XY-8, and CPMG sequences from top to bottom, each with its error-burden heatmap and metric box. GHZ-state fidelity, measured at the storage-window exit, and global purity both rise above the undecoupled baseline for the decoupled sequences.

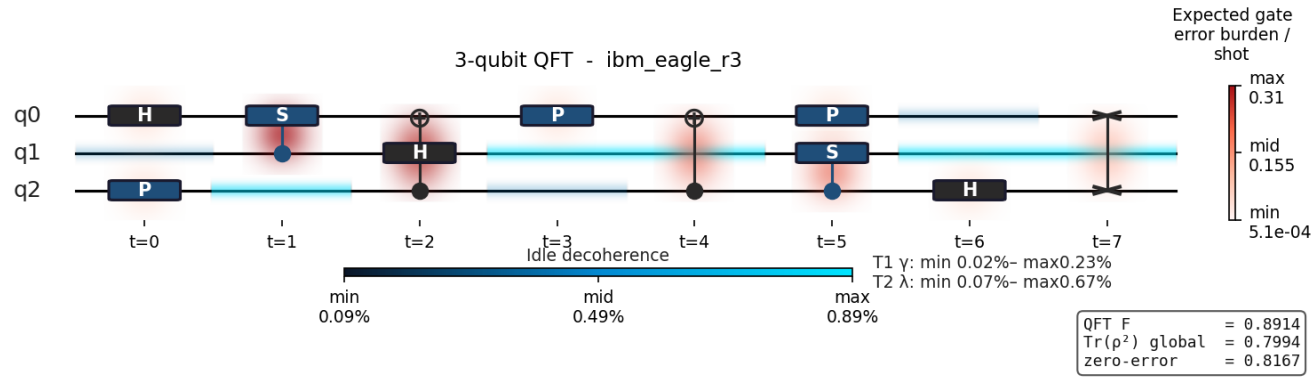

Figure 7: Aggregate error heatmap for the faithful (non-Clifford) 3-qubit QFT under the `ibm_eagle_r3` T_1/T_2 hardware profile (Pauli-twirled), averaged over 600 trajectory shots. **Gate halos** (red scale) reflect downstream propagation burden (Eq. 1); **idle halos** (blue scale) reflect T_1/T_2 decoherence on idling qubits. The metric box reports the trajectory fidelity F, global purity $\mathrm{Tr}(\rho^2)$, and zero-error shot fraction. Per-gate error rates are sampled exactly; the burden walk evaluates propagation under Clifford conjugation and treats the non-Clifford CS and $P(\pm\pi/8)$ gates as transparent, so the forward-spread halo component is approximate on those gates.

production-grade simulators are highly optimized for computational throughput, they often lack the intuitive, visual feedback necessary for students and early-career researchers to fully grasp how errors propagate through a circuit. NoisiQ addresses this educational and practical gap by providing a unified framework for noise-aware simulation and comprehensive visual diagnostics.

At its core, NoisiQ employs a dynamic routing architecture that automatically selects the most efficient simulation backend by leveraging Stim for highly scalable Clifford and Pauli-twirled circuits, and Qiskit Aer or a custom trajectory backend for exact density-matrix simulation of universal circuits. This flexibility allows users to seamlessly scale from small, exact physical models to large-scale quantum error correction demonstrations. By integrating pre-loaded hardware profiles, NoisiQ enables straightforward cross-platform benchmarking, allowing users to compare the effects of T_1 relaxation, T_2 dephasing, coherent over-rotations, and SPAM errors across leading quantum architectures like IBM superconducting processors and trapped-ion systems from Quantinuum and IonQ.

Furthermore, NoisiQ's suite of visualization tools, including step-by-step error trajectory animations, aggregate error heatmaps, and an interactive diagnostic interface, transforms abstract noise parameters into observable, circuit-level consequences. Coupled with built-in metrics and noise suppression techniques like dynamical decoupling, NoisiQ empowers users to rapidly prototype, test, and visualize error mitigation strategies.

Ultimately, NoisiQ serves as both a powerful research sandbox and an accessible educational instrument. By making the connection between physical noise models and algorithmic performance transparent, NoisiQ helps cultivate a deeper physical intuition for quantum computing, paving the way for more robust algorithm design in the presence of hardware noise.

Future Work

While NoisiQ currently provides a foundation for noise-aware simulation and visualization, there are several key areas for future development, primarily aimed at increasing its accessibility and physical accuracy.

First, we plan to expand the library's hardware integration by introducing more complex physical noise models. Real quantum hardware is plagued by architecture-specific phenomena, and as we expand NoisiQ to encompass more hardware modalities and profiles, we will incorporate these effects into our simulation framework. Future updates will focus on implementing detailed models for leakage (where qubits escape the computational subspace into higher energy states like $|2\rangle$), non-Markovian noise, and time-dependent crosstalk. By continuously refining these models, NoisiQ will further bridge the gap between simulation and the realities of quantum hardware.

Second, to support broader adoption within the quantum research and education communities, we aim to package NoisiQ for standard distribution. Currently, the library must be cloned and run locally. Transitioning the project to a fully packaged Python library installable via `pip` will streamline the setup process, allowing users to integrate NoisiQ into their existing Jupyter Notebook workflows and research pipelines.

Finally, we would like to build a fully interactive, web-based GUI for NoisiQ. Inspired by other educational tools like Quirk [23], this web application would allow users to construct quantum circuits using a drag-and-drop interface. As users build their circuits, the backend would dynamically run NoisiQ's simulations and render the heatmaps, density matrices, and error-propagation ani-

mations in real-time directly in the browser. This would drastically lower the barrier to entry, providing students and early researchers with a code-free sandbox to build physical intuition for quantum error correction and noise propagation.

Acknowledgments

We would like to express our gratitude to our advisors, Dr. Mohamad Niknam and Dr. Richard Ross, for their guidance, feedback, and support throughout the development of this project. We thank the UCLA I-Corp program, particularly Professor Farhad Rostamian, for the invaluable experience of teaching us how to pitch ourselves and our work. Additionally, we thank the members of the 2026 UCLA Master of Quantum Science and Technology (MQST) cohort for their insightful discussions and testing of early software iterations. Finally, we acknowledge the foundational work of the open-source quantum software community. NoisiQ's development was made possible by leveraging established frameworks, particularly IBM's Qiskit [8] and Craig Gidney's Stim [9].

References

[1] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.

[2] P. Krantz, M. Kjaergaard, F. Yan, T. P. Orlando, S. Gustavsson, and W. D. Oliver, "A quantum engineer's guide to superconducting qubits," *Applied Physics Reviews*, vol. 6, no. 2, p. 021318, 2019.

[3] M. Kjaergaard, M. E. Schwartz, J. Braumüller, P. Krantz, J. I.-J. Wang, S. Gustavsson, and W. D. Oliver, "Superconducting qubits: Current state of play," *Annual Review of Condensed Matter Physics*, vol. 11, pp. 369–395, 2020.

[4] F. Arute, K. Arya, R. Babbush, *et al.*, "Quantum supremacy using a programmable superconducting processor," *Nature*, vol. 574, no. 7779, pp. 505–510, 2019.

[5] IBM Quantum, "IBM Quantum debuts the 1,121-qubit Condor processor." `https://www.ibm.com/quantum/blog/quantum-roadmap-2033`, 2023. Announced at the IBM Quantum Summit 2023. Accessed: June 2026.

[6] Z. Cai and S. C. Benjamin, "Constructing smaller Pauli twirling sets for arbitrary error channels," *Scientific Reports*, vol. 9, no. 1, p. 11281, 2019.

[7] A. Hashim, R. K. Naik, A. Morvan, J.-L. Ville, B. Mitchell, J. M. Kreikebaum, M. Davis, E. Smith, C. Iancu, K. P. O'Brien, I. Hincks, J. J. Wallman, J. Emerson, and I. Siddiqi, "Randomized compiling for scalable quantum computing on a noisy superconducting quantum processor," *Physical Review X*, vol. 11, no. 4, p. 041039, 2021.

[8] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, "Quantum computing with Qiskit," *arXiv preprint arXiv:2405.08810*, 2024.

[9] C. Gidney, "Stim: a fast stabilizer circuit simulator," *Quantum*, vol. 5, p. 497, 2021.

[10] A. W. Cross, L. S. Bishop, J. A. Smolin, and J. M. Gambetta, "Open quantum assembly language," *arXiv preprint arXiv:1707.03429*, 2017.

[11] IBM Quantum, "IBM Quantum Platform." `https://quantum.cloud.ibm.com/computers`, 2026. Accessed: June 2026.

[12] M. AbuGhanem, "IBM quantum computers: Evolution, performance, and future directions," *arXiv preprint arXiv:2410.00916*, 2024.

[13] EmergentMind, "Quantinuum's H2 Quantum Processor." `https://www.emergentmind.com/topics/quantinuum-s-h2-quantum-processor`, 2025. Accessed: June 2026.

[14] Quantinuum, "Accessing hardware specification data." `https://docs.quantinuum.com/systems/user_guide/hardware_user_guide/benchmarks/hardware_data.html`, 2025. Accessed: June 2026.

[15] IonQ, "IonQ Forte Enterprise." `https://www.ionq.com/quantum-systems/forte-enterprise`, 2024. Accessed: June 2026.

[16] IonQ, "IonQ forte: First configurable quantum computer." `https://www.ionq.com/resources/ionq-forte-first-configurable-quantum-computer`, 2023. Accessed: June 2026.

[17] M. Van den Nest, "Classical simulation of quantum computation, the Gottesman-Knill theorem, and slightly beyond," *arXiv preprint arXiv:0811.0898*, 2008.

[18] J. J. Wallman and J. Emerson, "Noise tailoring for scalable quantum computation via randomized compiling," *Physical Review A*, vol. 94, no. 5, p. 052325, 2016.

[19] J. Emerson, M. Silva, O. Moussa, C. Ryan, M. Laforest, J. Baugh, D. G. Cory, and R. Laflamme, "Symmetrized characterization of noisy quantum processes," *Science*, vol. 317, no. 5846, pp. 1893–1896, 2007.

[20] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge, UK: Cambridge University Press, 2000.

[21] M. A. Nielsen, "A simple formula for the average gate fidelity of a quantum dynamical operation," *Physics Letters A*, vol. 303, no. 4, pp. 249–252, 2002.

[22] L. Viola, E. Knill, and S. Lloyd, "Dynamical decoupling of open quantum systems," *Physical Review Letters*, vol. 82, no. 12, pp. 2417–2421, 1999.

[23] C. Gidney, "Quirk: A drag-and-drop quantum circuit simulator." `https://algassert.com/quirk`, 2016. Accessed: June 2026.

[24] IonQ Staff, "IonQ aria: Practical performance." `https://www.ionq.com/resources/ionq-aria-practical-performance`, 2025. Accessed: June 2026.

[25] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (STOC)*, pp. 212–219, Association for Computing Machinery, 1996.

[26] D. Deutsch and R. Jozsa, "Rapid solution of problems by quantum computation," *Proceedings of the Royal Society of London. Series A: Mathematical and Physical Sciences*, vol. 439, no. 1907, pp. 553–558, 1992.

Hardware Noise Profiles

To facilitate accurate, cross-platform benchmarking, NoisiQ includes several pre-loaded hardware profiles. These profiles map published device specifications directly into NoisiQ's noise channels, including T_1 relaxation, T_2 dephasing, stochastic gate errors, coherent overrotations, and architecture-specific crosstalk. The parameters for the currently supported devices are detailed below.

.1 IBM Heron r3 [11, 12]

The Heron processor features a 156-qubit heavy-hexagonal lattice. Unlike previous generations, it utilizes tunable couplers that significantly suppress ZZ crosstalk.

- **Coherence Times:** $T_1 = 269\,\mu$s, $T_2 = 322\,\mu$s
- **Gate Infidelities:** Single-qubit: 1.6×10^{-4}, Two-qubit: 3×10^{-3}
- **Gate Durations:** Single-qubit: 50 ns, Two-qubit: 200 ns
- **SPAM Error:** 3.5×10^{-3}

.2 IBM Eagle r3

The Eagle processor features a 127-qubit heavy-hexagonal lattice using fixed-frequency transmon qubits and fixed couplers, resulting in higher residual ZZ coupling compared to Heron.

- **Coherence Times:** $T_1 = 263\,\mu$s, $T_2 = 147\,\mu$s
- **Gate Infidelities:** Single-qubit: 5.49×10^{-4}, Two-qubit: 1.16×10^{-2}
- **Gate Durations:** Single-qubit: 50 ns, Two-qubit: 561 ns
- **SPAM Error:** 1.35×10^{-2}

.3 Quantinuum H2-1 [13, 14]

The H2-1 is a 56-qubit trapped-ion device utilizing a Quantum Charge-Coupled Device (QCCD) architecture. It features all-to-all connectivity achieved by physically shuttling ions between four parallel gate zones.

- **Coherence Times:** $T_1 = 100.0$ s, $T_2 = 1.0$ s
- **Gate Infidelities:** Single-qubit: 3×10^{-5}, Two-qubit: 1.5×10^{-3}
- **Gate Durations:** Single-qubit: $5\,\mu$s, Two-qubit: $150\,\mu$s
- **SPAM Error:** 2×10^{-3}

.4 IonQ Forte [15, 16]

The Forte is a 36-qubit trapped-ion system that maintains a single linear chain of ions. It achieves all-to-all connectivity via individual optical addressing rather than physical shuttling.

- **Coherence Times:** $T_1 = 50.0$ s, $T_2 = 1.0$ s
- **Gate Infidelities:** Single-qubit: 2×10^{-4}, Two-qubit: 4×10^{-3}
- **Gate Durations:** Single-qubit: $135\,\mu$s, Two-qubit: $600\,\mu$s
- **SPAM Error:** 5×10^{-3}

.5 IonQ Aria [24]

The Aria is an earlier generation 25-qubit (21 production) trapped-ion system, sharing a similar linear chain architecture to the Forte but with slightly higher gate infidelities.

- **Coherence Times:** $T_1 = 50.0$ s, $T_2 = 1.0$ s
- **Gate Infidelities:** Single-qubit: 6×10^{-4}, Two-qubit: 6×10^{-3}
- **Gate Durations:** Single-qubit: $135\,\mu$s, Two-qubit: $600\,\mu$s
- **SPAM Error:** 3.9×10^{-3}

Supporting Examples / Code

Grover's Algorithm

Grover's search amplifies the amplitude of a marked computational-basis state through repeated application of an oracle and a diffusion operator [25]. The NoisiQ example implements a 3-qubit search for the marked state $|111\rangle$ over two Grover iterations. Each iteration applies the phase oracle — a doubly-controlled Z (CCZ) on the three qubits, which flips the phase of $|111\rangle$ — followed by the diffuser ($H^{\otimes 3}\,X^{\otimes 3}\,\mathrm{CCZ}\,X^{\otimes 3}\,H^{\otimes 3}$), which reflects the state about the uniform superposition. For three qubits the optimal iteration count is $\lfloor \frac{\pi}{4}\sqrt{8} \rfloor = 2$, so this circuit reaches close to peak success probability in the noiseless limit. The circuit is non-Clifford (the CCZ gates produce non-stabilizer intermediate states), so it is simulated with the statevector-based `TrajectoryBackend`; the aggregate error-burden pass is run via `run_aggregate`, which returns the same `AggregateResult` dataclass as `ManyShotRunner` together with the averaged density matrix. As in the GHZ example, the dominant per-shot error burden falls on the two-qubit entangling layers — here the four CCZ gadgets (one oracle and one diffuser reflection per iteration) — visible as the concentrated halos in Figure 8, while the single-qubit H/X layers contribute comparatively little. Under an `ibm_eagle_r3` hardware noise profile aggregated over 2 000 shots, the noisy state retains an ideal-state fidelity of $F = \langle\psi_{\text{ideal}}|\rho|\psi_{\text{ideal}}\rangle = 0.8383$ with the noiseless Grover output, and the probability of measuring the marked state is $P(|111\rangle) = 0.7944$, compared with ≈ 0.94 in the noiseless limit. The per-qubit marginal purities are $\mathrm{Tr}(\rho_q^2) = 0.814,\ 0.821,\ 0.819$ for q_0, q_1, q_2, reflecting the residual mixing introduced across the two iterations. Note that NoisiQ applies the CCZ as a native three-qubit gate and assigns it the model's two-qubit error rate; on hardware the CCZ would be compiled into multiple two-qubit gates, so the simulated error burden on the oracle and diffuser is a lower bound relative to a fully compiled implementation.

Deutsch–Jozsa Algorithm

The Deutsch–Jozsa algorithm determines whether a Boolean function $f : \{0,1\}^n \to \{0,1\}$ is constant or balanced in a single oracle query, where any classical deterministic algorithm requires up to $2^{n-1}+1$ queries in the worst case [26]. The NoisiQ example implements the 3-qubit case (two input qubits q_0, q_1 and one ancilla q_2) with a balanced oracle. The ancilla is prepared in $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$ by an X followed by a Hadamard, so that the oracle's bit-flips act as phase kickbacks on the input register; the balanced oracle is realized as $\mathrm{CNOT}(q_0 \to q_2)\,\mathrm{CNOT}(q_1 \to q_2)$, and a final Hadamard layer on the input qubits converts the kicked-back phases into the measured outcome. For a balanced oracle the input register reads $|11\rangle$ with certainty in the noiseless limit; a constant oracle would instead yield $|00\rangle$.

The circuit is entirely Clifford (H, X, CNOT), so `ManyShotRunner` routes the aggregate error-burden pass directly to the Stim backend, while a separate `TrajectoryBackend` pass supplies the density matrix for the fidelity metrics. A noiseless self-check confirms $P(|11\rangle) = 1.000$ before any noise is applied, validating the circuit construction from first principles. Because the circuit is shallow (depth five, with only the two oracle CNOTs as entangling gates), it is comparatively robust to noise: the dominant per-shot error burden falls on the two CNOT layers, with idle T_1/T_2 decoherence accumulating on the input qubits during the oracle, visible as the wire halos in Figure 9.

Under an `ibm_eagle_r3` hardware noise profile, the noisy state retains an ideal-state fidelity of $F = \langle\psi_{\text{ideal}}|\rho|\psi_{\text{ideal}}\rangle = 0.9400$ with the noiseless output, and the probability of correctly reading the balanced-oracle answer is $P(|11\rangle) = 0.9525$, compared with the deterministic $P(|11\rangle) = 1$ in the noiseless limit. The marked-state probability slightly exceeds the full-state fidelity because it marginalizes over the ancilla and the relative phase, forgiving errors that the full-state overlap penalizes. The per-qubit marginal purities are $\mathrm{Tr}(\rho_q^2) = 0.951,\ 0.956,\ 0.951$ for q_0, q_1, q_2. The high values across all metrics — in contrast to the deeper Grover circuit of Appendix .5 — reflect the shallow, low-entangling structure that makes Deutsch–Jozsa a forgiving benchmark for a near-term noise model.

Quantum Phase Estimation

Quantum phase estimation (QPE) estimates the eigenphase φ of a unitary U, given one of its eigenstates, by writing a binary approximation of $\theta = \varphi/2\pi$ into a counting register through controlled applications of U followed by an inverse quantum Fourier transform [20]. The NoisiQ example implements the three-counting-qubit case (counting qubits q_0, q_1, q_2 and one target q_3) for $U = P(\varphi)$ with $\varphi = \pi/4$, so the true phase is $\theta = 1/8 = 0.001_2$, exactly representable in three bits. The target qubit is prepared in $|1\rangle$ — an eigenstate of $P(\varphi)$ with eigenvalue $e^{i\varphi}$ — by an X gate, the counting register is placed in uniform superposition by a Hadamard layer, and each counting qubit q_k controls $U^{2^k} = P(\varphi\,2^k)$ on the target, kicking the phase $e^{2\pi i\theta 2^k}$ back onto the control. An inverse QFT on the counting register then converts these accumulated phases into the measured binary estimate. Because the encoding assigns q_0 bit-weight 2^0 (least significant) and q_2 bit-weight 2^2 (most significant), the inverse QFT is ordered with q_2 as the most-significant qubit to match; the correct readout is the counting-register state $|100\rangle_{q_0 q_1 q_2}$, i.e. integer 1 and $\hat{\theta} = 1/8$.

The controlled-P rotations are non-Clifford, so the heatmap is produced from a Clifford stand-in (each $P(\theta)$ replaced by $S/S^\dagger$ with the CNOT topology preserved) routed through `ManyShotRunner` on the Stim backend,

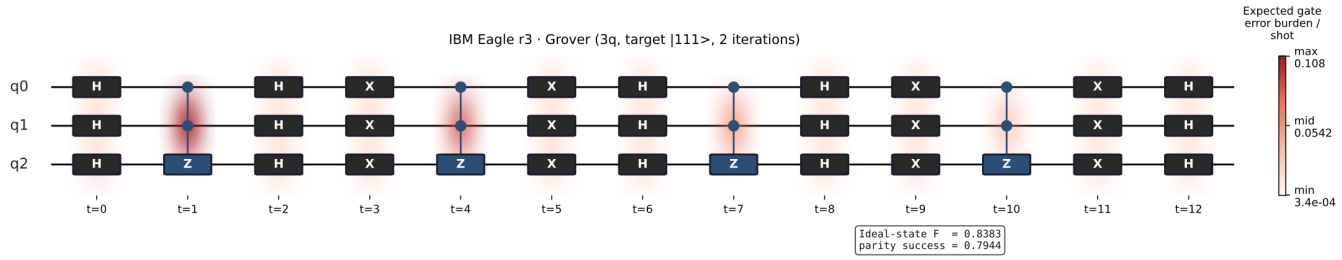

Figure 8: Aggregate error heatmap for a 3-qubit Grover search targeting $|111\rangle$ over two iterations, under an `ibm_eagle_r3` hardware noise profile aggregated over 2 000 shots. Red gate and wire halos encode the expected per-shot gate error burden, which concentrates on the two-qubit CCZ gadgets (the oracle and diffuser reflections) and is minimal on the single-qubit H/X layers. The metric box reports the ideal-state fidelity $F = 0.8383$ (overlap of the noisy state with the full noiseless output) and the marked-state success probability $P(|111\rangle) = 0.7944$ (noiseless ≈ 0.94). The per-qubit marginal purities after tracing out the other qubits are $\mathrm{Tr}(\rho_q^2) \approx 0.81$–$0.82$.

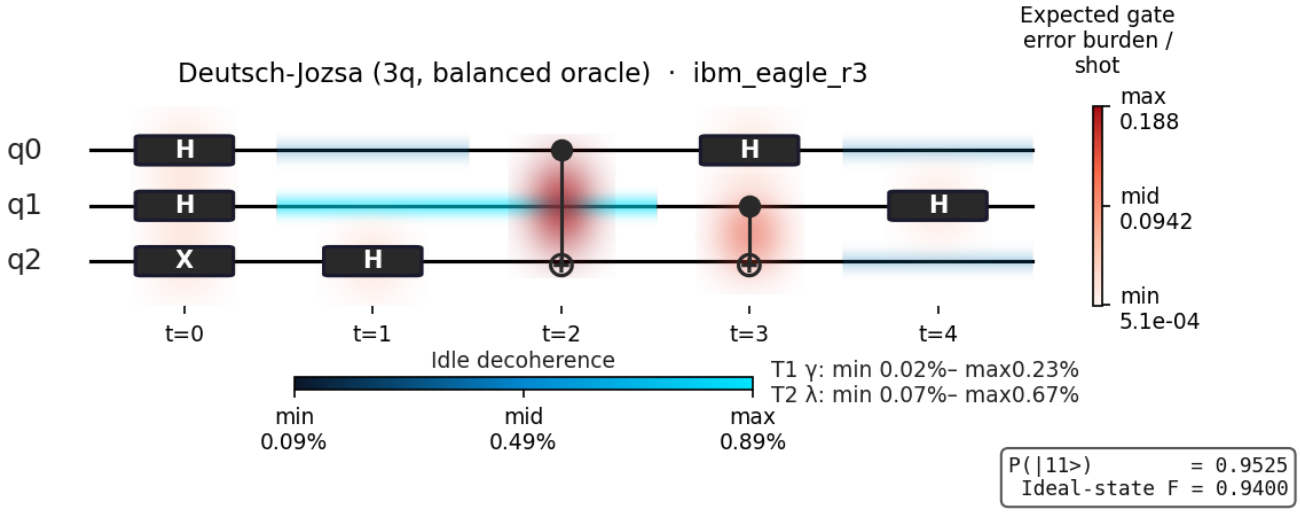

Figure 9: Aggregate error heatmap for the 3-qubit Deutsch–Jozsa algorithm with a balanced oracle, under an `ibm_eagle_r3` hardware noise profile aggregated over 2 000 shots. Red gate and wire halos encode the expected per-shot gate error burden, which concentrates on the two oracle CNOT layers ($t = 2$, $t = 3$); the blue gradient shows idle T_1/T_2 decoherence accumulating on the input qubits. The metric box reports the ideal-state fidelity $F = 0.9400$ and the marked-state success probability $P(|11\rangle) = 0.9525$ (noiseless $P(|11\rangle) = 1$).

while the fidelity and success-probability metrics come from a separate `TrajectoryBackend` pass on the real circuit under a coherent error model. A noiseless self-check confirms $P(|100\rangle) = 1.000$ before any noise is applied, validating both the circuit construction and the inverse-QFT qubit ordering from first principles. Each controlled-U^{2^k} and each inverse-QFT controlled-phase gadget decomposes into a pair of CNOTs, making QPE the most entangling-gate-dense of the appendix examples; the dominant per-shot error burden therefore concentrates on these CNOT layers, with idle T_1/T_2 decoherence accumulating on the counting qubits across the long gadget sequence, visible as the gate and wire halos in Figure 10.

Under an `ibm_eagle_r3` hardware noise profile (heatmap aggregated over 2 000 shots; metrics from a 600-trajectory pass), the noisy state retains an ideal-state fidelity of $F = \langle\psi_{\text{ideal}}|\rho|\psi_{\text{ideal}}\rangle = 0.7414$ with the noiseless output, and the probability of reading the correct phase is $P(|100\rangle) = 0.7580 \pm 0.0175$, compared with the exact $P(|100\rangle) = 1$ in the noiseless limit. As in the Deutsch–Jozsa example of Appendix .5, the readout success probability slightly exceeds the full-state fidelity because it marginalizes over the target qubit and the relative phases, forgiving errors that the full-state overlap penalizes. The per-qubit marginal purities are $\mathrm{Tr}(\rho_q^2) = 0.851,\ 0.786,\ 0.751,\ 0.923$ for q_0, q_1, q_2, q_3, the lower counting-register values reflecting the residual mixing introduced across the deep controlled-phase and inverse-QFT sequence. The uniformly lower metrics relative to Grover (Appendix .5) and Deutsch–Jozsa (Appendix .5) are consistent with QPE's greater circuit depth and entangling-gate count.

Code Listings

The examples in this appendix and the figures throughout the paper share a single, uniform workflow. A user (i) builds a circuit, (ii) attaches a noise model, (iii) fills idle gaps so decoherence accrues on waiting qubits, (iv) runs a backend, and (v) reads out a metric or renders a diagnostic. The listings below give that workflow in full and then show the two ways NoisiQ accepts a noise model. The per-algorithm circuits in the rest of this appendix differ only in step (i); steps (ii)–(v) are identical to Listing 1. Each listing is self-contained and runnable against the public `noisiq` API; for brevity we import only the symbols each example actually uses.

```python
n = 4
circuit = nq.Circuit(n_qubits=n, name="ghz")
circuit.h(0)                          # Hadamard on q0
for q in range(n - 1):
    circuit.cnot(q, q + 1)            # CNOT cascade spreads
↪ entanglement

# (2) NOISE MODEL from a calibrated hardware profile (see Listing
↪ 2 for the
#     full set of options, and Listing 3 for a hand-built
↪ alternative).
profile = nq.noise.get_hardware("ibm_eagle_r3")

# (3) FILL IDLE slots: insert IDLE ops on qubits that are waiting,
↪  so the
#     noise model applies T1/T2 decoherence during idle time. gate
↪ _times
#     supplies the physical duration of each gate.
circuit = fill_idle_with_identities(circuit, profile.gate_times)

# Build the per-operation noise model AFTER filling, so the
↪ freshly added
# IDLE ops also receive a channel. "pauli_twirl" expresses every
↪ channel as
# stochastic X/Y/Z, which the Pauli-frame backend and heatmap
↪ require.
noise = profile.to_noise_model(circuit, mode="t2", representation=
↪ "pauli_twirl")

# (4) RUN a backend. TrajectoryBackend is the exact density-matrix
↪  engine
#     (<= 13 qubits); it returns a SimulationResult whose final_
↪ state is the
#     averaged density matrix rho.
result = TrajectoryBackend().run(circuit, noise_model=noise, n_
↪ shots=2000, seed=42)
rho = result.final_state

# (5) METRICS. The metric helpers return a MetricReport; the
↪ number is .value.
fidelity = ghz_state_fidelity(rho, n).value     # F = <GHZ|rho|GHZ
↪ >
purity   = global_purity(rho).value             # Tr(rho^2): 1.0 =
↪  pure
print(f"GHZ fidelity = {fidelity:.4f},  purity = {purity:.4f}")
```

Listing 1: The NoisiQ pipeline. Every example in the paper is this template with a different circuit builder in step (1). Here a GHZ circuit is run under a hardware noise profile and its state fidelity and global purity are reported.

NoisiQ accepts a noise model in two forms. The first, used for every hardware example in the paper, is a `HardwareProfile` retrieved from the built-in registry. Calling `to_noise_model()` on the profile returns a dictionary mapping each operation index to a noise channel. The `representation` argument selects how coherent and idle errors are expressed: `"pauli_twirl"` produces stochastic X/Y/Z channels (required by the Stim/`ManyShotRunner` path and the error heatmap), while `"coherent"` keeps idle dephasing as a genuine per-shot rotation, which is the representation that lets dynamical decoupling refocus it (Section 5.2).

```python
import noisiq as nq
from noisiq.backends import TrajectoryBackend
from noisiq.noise import fill_idle_with_identities
from noisiq.results import ghz_state_fidelity
from noisiq.visualization.density_matrix import global_purity

# (1) BUILD a circuit with the fluent builder. Each call returns
↪ the
#     circuit, so calls can be chained. t= pins a gate to a time
↪ layer;
#     omit it to let NoisiQ schedule the gate into the next free
↪ slot.
```

```python
import noisiq as nq

# Discover available calibrated profiles (IBM Eagle/Heron, IonQ,
↪ Quantinuum...).
print(nq.noise.list_hardware())

profile = nq.noise.get_hardware("ibm_eagle_r3")
print(profile.t1, profile.t2)         # median T1, T2 in seconds

# representation="pauli_twirl": stochastic X/Y/Z -> Pauli-frame +
↪ heatmap.
noise_twirl = profile.to_noise_model(circuit, mode="t2",
```

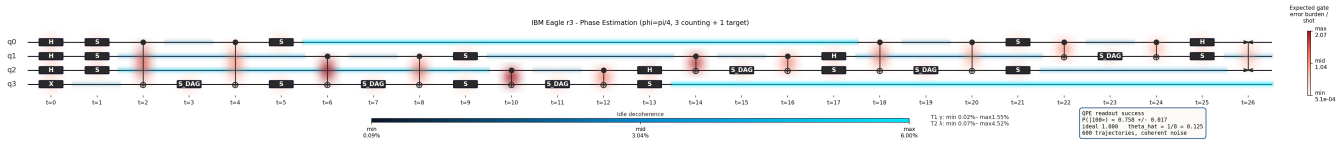

Figure 10: Aggregate error heatmap for the three-counting-qubit quantum phase estimation circuit ($U = P(\pi/4)$, $\theta = 1/8$), under an `ibm_eagle_r3` hardware noise profile (heatmap aggregated over 2 000 shots). Red gate and wire halos encode the expected per-shot gate error burden, which concentrates on the CNOT pairs inside the controlled-U^{2^k} and inverse-QFT controlled-phase gadgets; the blue gradient shows idle T_1/T_2 decoherence on the counting qubits. The metric box reports the ideal-state fidelity $F = 0.7414$, the readout success probability $P(|100\rangle) = 0.7580 \pm 0.0175$ from a 600-trajectory pass (noiseless $P(|100\rangle) = 1$), and the recovered phase $\hat{\theta} = 1/8$. The counting bitstring is written $q_0 q_1 q_2$ with q_0 the least-significant bit.

```
                                        representation="pauli_twirl")

# representation="coherent": idle-Z drift stays a real rotation
↪ that DD can
# refocus -> use this for the fidelity/purity pass in the DD
↪ example.
noise_coherent = profile.to_noise_model(circuit, mode="t2",
                                        representation="coherent")
```

Listing 2: Passing a noise model from a hardware profile. `list_hardware()` enumerates the registry; the chosen profile builds a per-operation channel dictionary.

The second form is an explicit dictionary the user constructs directly. A noise model is simply a mapping `{operation_index: channel}`, so any per-gate error structure can be specified by hand. This is the form used for the Bell parity-check figure (Section 3.2), where an X-biased Pauli channel gives the parity check a visible, interpretable job. The `PauliError` channel takes the three single-qubit error probabilities p_X, p_Y, p_Z directly.

```
from noisiq.ir.circuit import Operation
from noisiq.noise import PauliError

def pauli_noise_for_ops(circuit, p_1q=0.006, p_2q=0.014, x_bias
↪ =0.85):
    """Map each gate to a PauliError. Two-qubit gates get the
↪ larger rate;
    the non-X share is split evenly between Y and Z."""
    noise = {}
    for idx, op in enumerate(circuit.operations):
        if not isinstance(op, Operation):     # skip measurements,
↪  etc.
            continue
        p = p_2q if op.gate.num_qubits == 2 else p_1q
        noise[idx] = PauliError(p_x=p * x_bias,
                                p_y=p * (1 - x_bias) / 2,
                                p_z=p * (1 - x_bias) / 2)
    return noise

noise = pauli_noise_for_ops(circuit)          # ready to pass to
↪ any backend
```

Listing 3: A hand-built Pauli noise model. The model is a dict from operation index to a channel; here a larger error rate is assigned to two-qubit gates than to single-qubit gates, with an X bias.

GHZ state (Section 5.1). The GHZ circuit is entirely Clifford, so the aggregate error-burden pass that colours the heatmap is routed to the Stim backend through `ManyShotRunner`, while a separate `TrajectoryBackend` pass supplies the density matrix for the fidelity and purity metrics. Both passes use the same circuit and the same hardware profile.

```
import noisiq as nq
from noisiq.backends import ManyShotRunner, TrajectoryBackend
from noisiq.noise import fill_idle_with_identities
from noisiq.results import ghz_state_fidelity

def build_ghz_circuit(n):
    c = nq.Circuit(n_qubits=n, name=f"ghz_{n}q")
    c.h(0)                                    # superpose q0
    for q in range(n - 1):
        c.cnot(q, q + 1)                      # entangle the cascade
    return c

n = 13
circuit = build_ghz_circuit(n)
profile = nq.noise.get_hardware("ibm_eagle_r3")
circuit = fill_idle_with_identities(circuit, profile.gate_times)
noise   = profile.to_noise_model(circuit, mode="t2",
↪ representation="pauli_twirl")

# Clifford -> Stim path: per-gate error statistics for the heatmap
↪ .
aggregate = ManyShotRunner().run(circuit, n_shots=4096, noise_
↪ config=noise, seed=42)
print("zero-error fraction:", aggregate.zero_error_fraction)

# Density-matrix path: fidelity metric.
rho = TrajectoryBackend().run(circuit, noise_model=noise, n_shots
↪ =2000, seed=42).final_state
print("GHZ fidelity:", ghz_state_fidelity(rho, n).value)
```

Listing 4: Building and running the N-qubit GHZ example. The Clifford circuit is sent to Stim via `ManyShotRunner` for error statistics; `TrajectoryBackend` provides the density matrix for the reported metrics.

Bell parity check (Section 3.2). This circuit exercises mid-circuit measurement and classical feedback. A parity ancilla (q2) is measured and reset, and its outcome conditionally triggers a correction on q1 through `c_if`. Classical bits are allocated with `add_classical_register`, and `measure` writes an outcome into a named bit. The hand-built noise model of Listing 3 is used here so the parity repair has a visible effect.

```
import noisiq as nq
from noisiq.backends import ManyShotRunner, StimTableauBackend

c = nq.Circuit(n_qubits=3, name="bell_parity_check")
syndrome = c.add_classical_register("syndrome", 1)   # 1 classical
```

```
↪ bit

# Bell pair on q0, q1.
c.h(0, t=0)
c.cnot(0, 1, t=1)

# ZOZ1 parity into ancilla q2, then measure + reset it.
c.cnot(0, 2, t=2)
c.cnot(1, 2, t=3)
c.measure(2, syndrome[0], reset=True, t=4)

# Conditional repair: if the parity bit is 1, apply X to q1.
c.c_if(syndrome[0], value=1).x(1, t=5)

# noise = pauli_noise_for_ops(c)   # from Listing 3
runner = ManyShotRunner(StimTableauBackend())        # Clifford ->
↪ Stim
aggregate = runner.run(c, n_shots=4096, noise_config=noise, seed
↪ =42)
```

Listing 5: The Bell parity-check circuit with a mid-circuit measurement (`measure(..., reset=True)`) and a conditional correction (`c_if(...).x(...)`). `syndrome[0]` is a classical bit allocated on the circuit.

A non-Clifford circuit (for example the QFT of Section 5.3) needs no change to this workflow: passing it to NoisiQ's `BackendSelector`, or calling `TrajectoryBackend().run_aggregate(...)` directly, detects the non-Clifford gates and selects the density-matrix backend automatically, while the noise-model and metric steps remain exactly as above.